Web Services CA 1

Julia Udvari

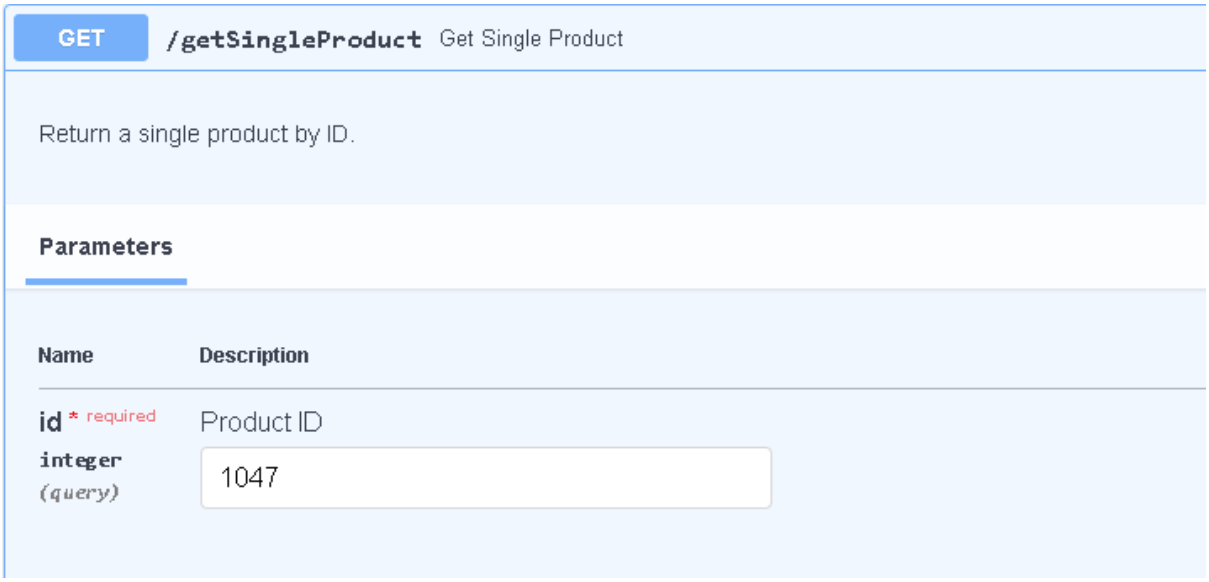
B00153580

01/04/2026

Contents

API Screenshots

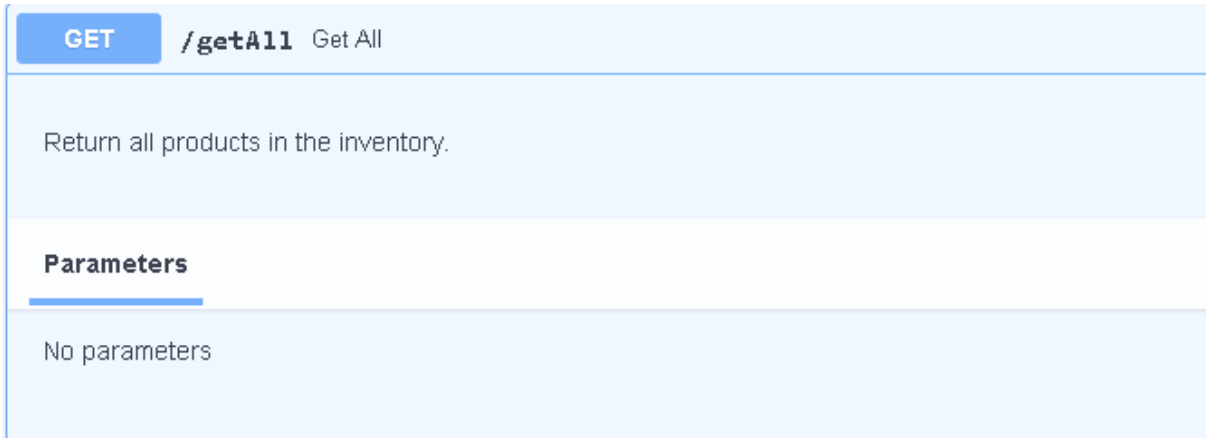
/getSingleProduct



Response body


```
{
  "_id": "69ccb16403293f42cd36cdfe",
  "ProductID": 1047,
  "Name": "ASUS ProArt Z790",
  "UnitPrice": 470,
  "StockQuantity": 6,
  "Description": "Creator-focused motherboard with Thunderbolt 4."
}
```


/getAll



Response body


```
[
  {
    "_id": "69ccb16403293f42cd36cdd0",
    "ProductID": 1001,
    "Name": "NVIDIA RTX 4090",
    "UnitPrice": 1599.99,
    "StockQuantity": 12,
    "Description": "High-end GPU with 24GB VRAM for 4K gaming."
  },
  {
    "_id": "69ccb16403293f42cd36cdd1",
    "ProductID": 1002,
    "Name": "AMD Ryzen 9 7950X",
    "UnitPrice": 549,
    "StockQuantity": 25,
    "Description": "16-core 32-thread enthusiast desktop processor."
  },
  {
    "_id": "69ccb16403293f42cd36cdd2",
    "ProductID": 1003,
    "Name": "Samsung 990 Pro 2TB",
    "UnitPrice": 179.99,
    "StockQuantity": 40,
    "Description": "PCIe Gen4 NVMe M.2 SSD with 7450MB/s reads."
  },
  {
    "_id": "69ccb16403293f42cd36cdd3",
    "ProductID": 1004,
```


/addNew

POST /addNew Add New

Add a new product to the inventory.

Parameters

No parameters

Request body required

Edit Value | Schema


```
{
  "ProductID": 2001,
  "Name": "Logitech MX Master 3S",
  "UnitPrice": 50.00,
  "StockQuantity": 78,
  "Description": "Wireless mouse"
}
```

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8000/addNew' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "ProductID": 2001,
  "Name": "Logitech MX Master 3S",
  "UnitPrice": 50.00,
  "StockQuantity": 78,
  "Description": "Wireless mouse"
}'
```

/deleteOne

DELETE `/deleteOne` Delete One

Delete a product by ID.

Parameters

Name	Description
id * required integer *(query)*	Product ID to delete 2001

Response body

```
{
  "message": "Product deleted successfully"
}
```

/startswith

Response body


```
[
  {
    "_id": "69ccb16403293f42cd36cdd0",
    "ProductID": 1001,
    "Name": "NVIDIA RTX 4090",
    "UnitPrice": 1599.99,
    "StockQuantity": 12,
    "Description": "High-end GPU with 24GB VRAM for 4K gaming."
  },
  {
    "_id": "69ccb16403293f42cd36cdd6",
    "ProductID": 1007,
    "Name": "NZXT H7 Flow Case",
    "UnitPrice": 129.99,
    "StockQuantity": 22,
    "Description": "Mid-tower ATX case with high-airflow mesh front."
  },
  {
    "_id": "69ccb16403293f42cd36cdd7",
    "ProductID": 1008,
    "Name": "Noctua NH-D15",
    "UnitPrice": 109.95,
    "StockQuantity": 45,
    "Description": "Dual-tower flagship CPU air cooler."
  },
  {
    "_id": "69ccb16403293f42cd36ce09",
    "ProductID": 1058,
```

/paginate

GET /paginate Paginate

Return products in batches of 10 between start and end ID.

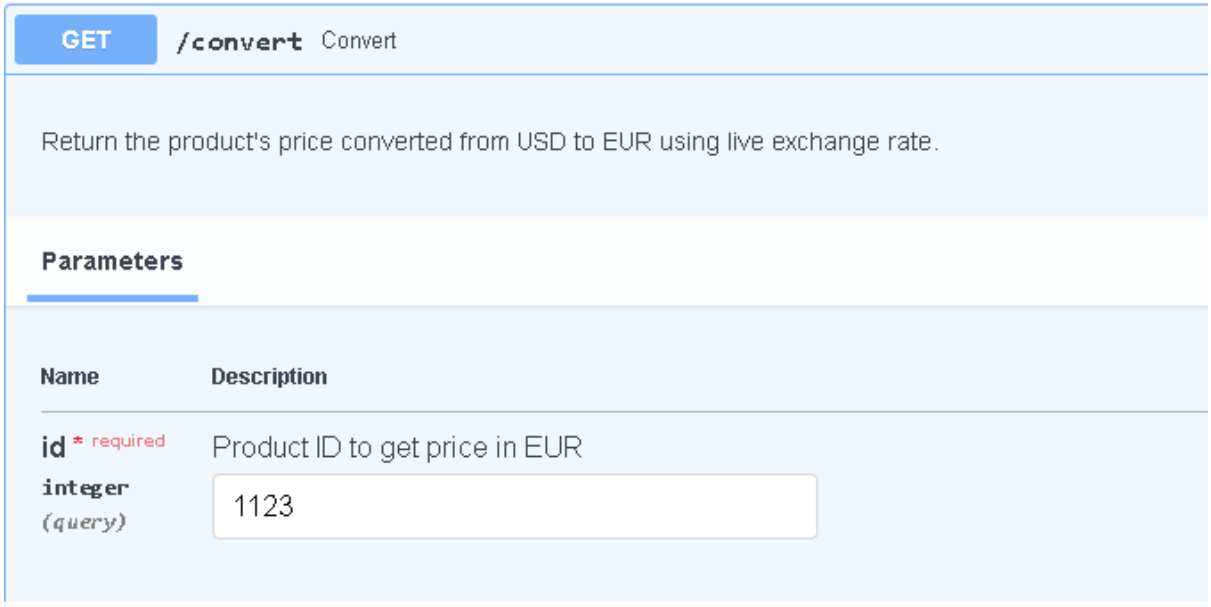
Parameters

Name	Description
startId * required integer *(query)*	Starting product ID 1050
endId * required integer *(query)*	Ending product ID 1080

Response body


```
[
  {
    "_id": "69ccb16403293f42cd36ce01",
    "ProductID": 1050,
    "Name": "Thermal Grizzly Kryonaut",
    "UnitPrice": 11.99,
    "StockQuantity": 150,
    "Description": "High-end thermal grease for overclocking."
  },
  {
    "_id": "69ccb16403293f42cd36ce02",
    "ProductID": 1051,
    "Name": "Sapphire Pulse RX 7800 XT",
    "UnitPrice": 499.99,
    "StockQuantity": 15,
    "Description": "Efficient 1440p gaming AMD GPU."
  },
  {
    "_id": "69ccb16403293f42cd36ce03",
    "ProductID": 1052,
    "Name": "Intel Core i5-12400F",
    "UnitPrice": 149,
    "StockQuantity": 45,
    "Description": "Best value budget gaming CPU (no iGPU)."
  },
  {
    "_id": "69ccb16403293f42cd36ce04",
    "ProductID": 1053,
```

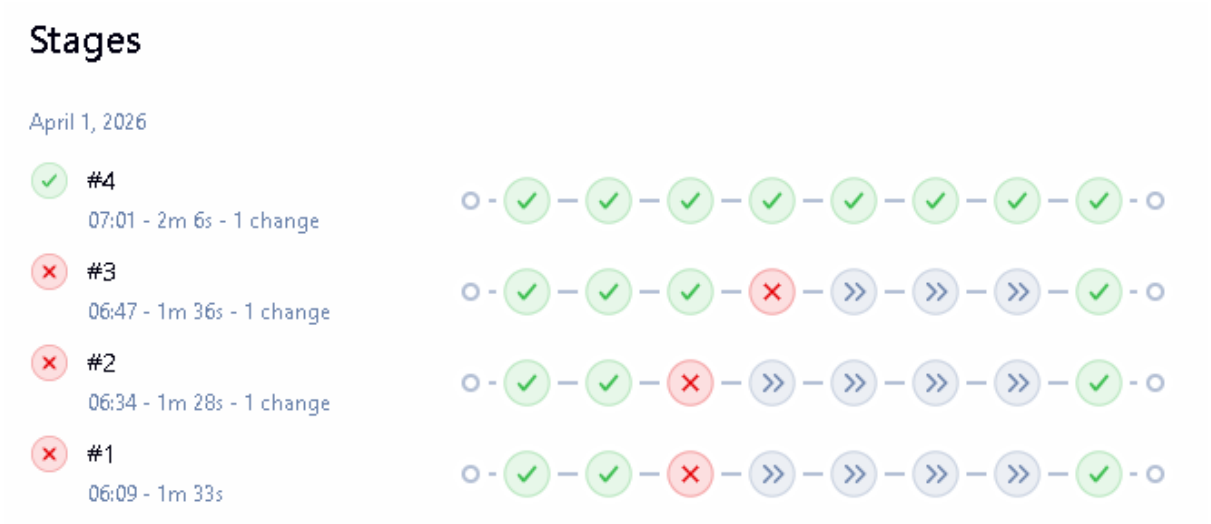
/convert



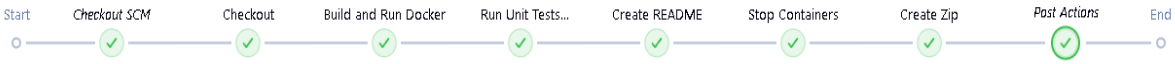
Response body


```
{
  "ProductID": 1123,
  "Name": "MSI MAG Z790 Tomahawk",
  "PriceUSD": 269.99,
  "PriceEUR": 234.82,
  "ExchangeRate": 0.86972
}
```


Jenkins Builds



Pipeline Overview for #4



End of Console Output

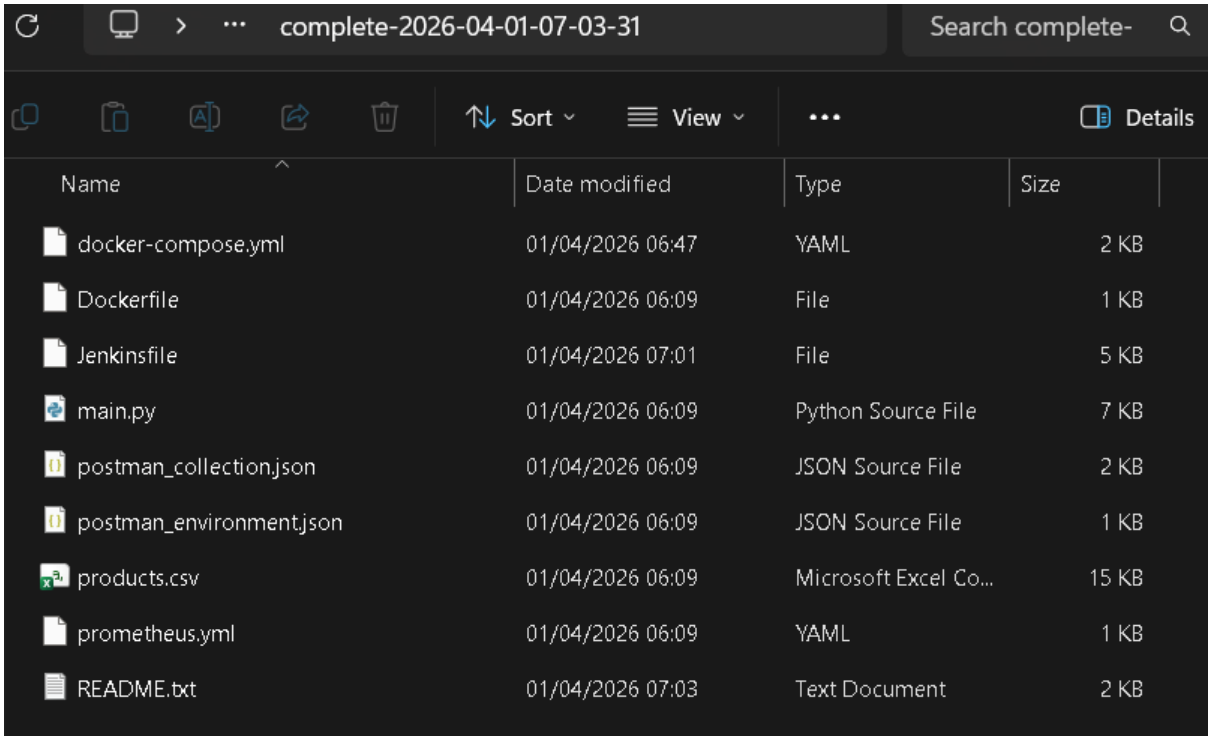
```
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Zip file Created

Console output

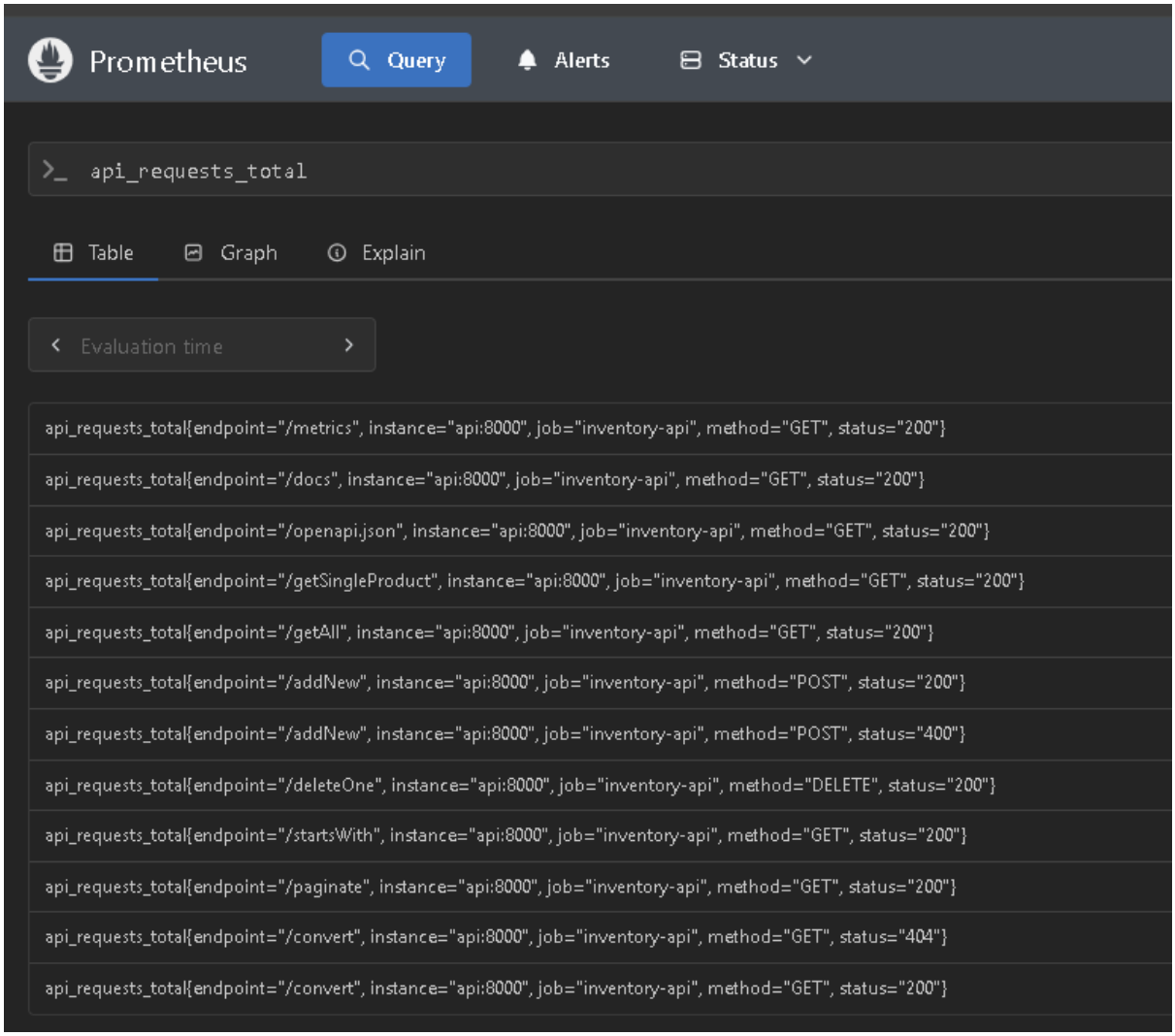
```
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Create Zip)
[Pipeline] powershell
Created: complete-2026-04-01-07-03-31.zip
[Pipeline] }
```

Zip contents

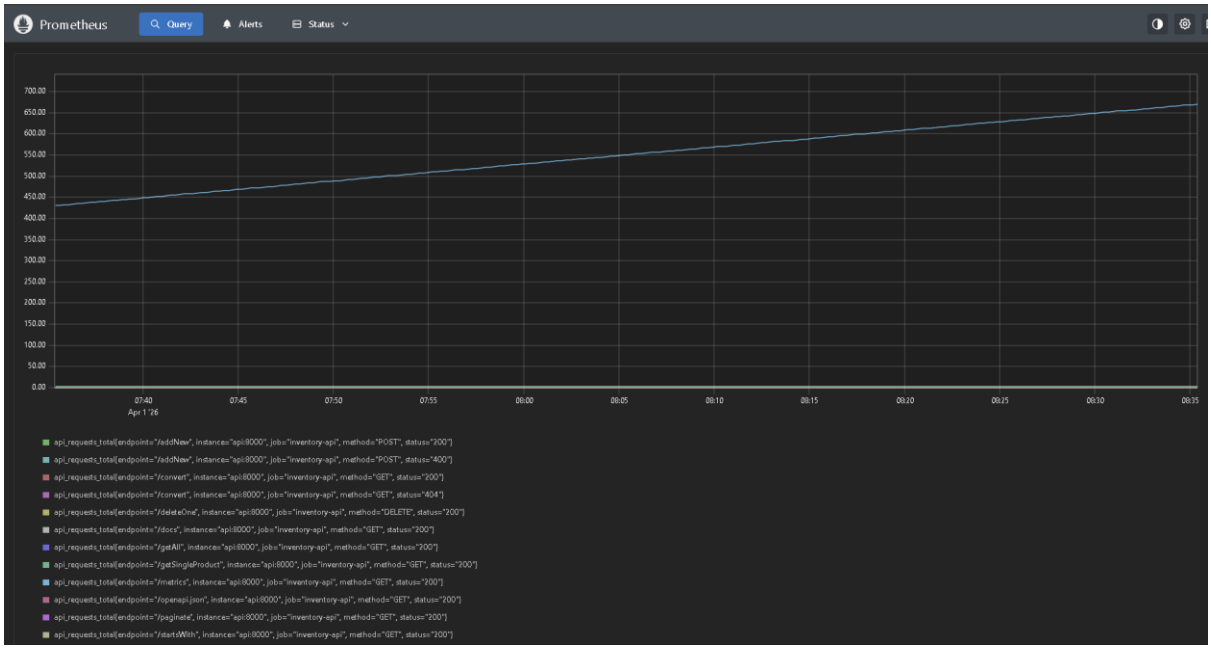


A-grade component

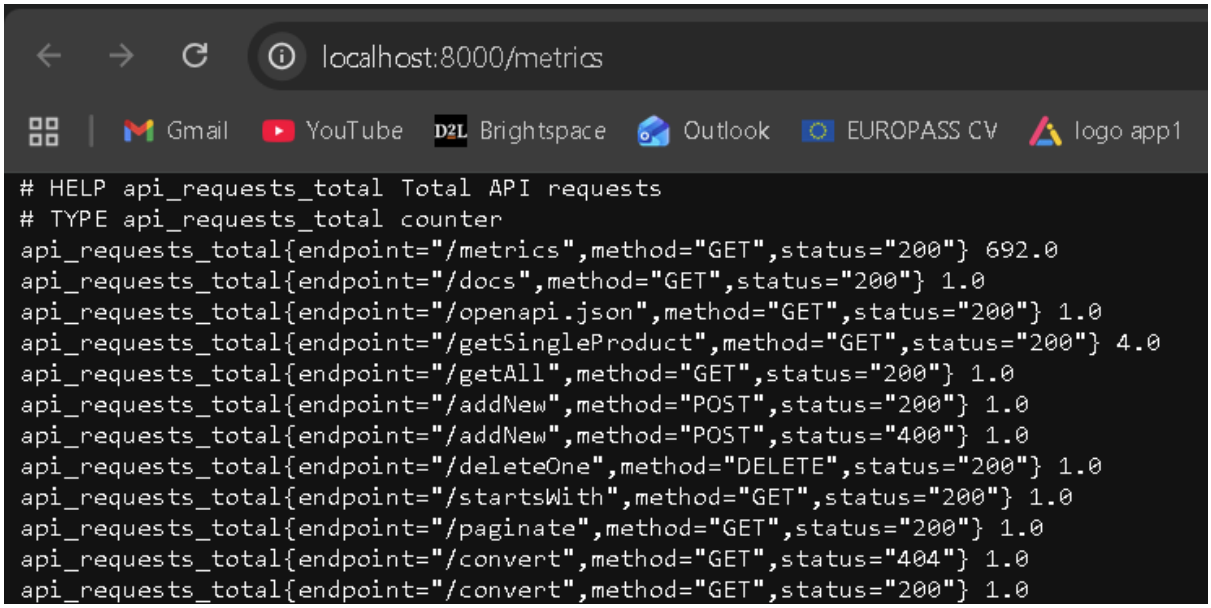
Prometheus table



Prometheus graph



Metrics endpoint



Appendix

Fast API Code

main.py:

```
import os
```

```
from contextlib import asynccontextmanager

import httpx
from fastapi import FastAPI, HTTPException, Query
from pydantic import BaseModel, Field
from pymongo import MongoClient
from prometheus_client import Counter, Histogram, generate_latest,
CONTENT_TYPE_LATEST
from fastapi.responses import Response

# MongoDB config
MONGODB_URI = os.getenv("MONGODB_URI", "mongodb://localhost:27017")
DATABASE_NAME = "inventory_db"
COLLECTION_NAME = "products"

# Prometheus metrics (API monitoring)
REQUEST_COUNT = Counter(
    "api_requests_total",
    "Total API requests",
    ["method", "endpoint", "status"]
)
REQUEST_LATENCY = Histogram(
    "api_request_duration_seconds",
    "API request latency in seconds",
    ["endpoint"]
)

# Pydantic models for validation
class Product(BaseModel):
    ProductID: int = Field(..., gt=0, description="Unique product identifier")
```

```
    Name: str = Field(..., min_length=1, max_length=200)
    UnitPrice: float = Field(..., ge=0)
    StockQuantity: int = Field(..., ge=0)
    Description: str = Field(..., min_length=1)

class ProductCreate(BaseModel):
    ProductID: int = Field(..., gt=0)
    Name: str = Field(..., min_length=1, max_length=200)
    UnitPrice: float = Field(..., ge=0)
    StockQuantity: int = Field(..., ge=0)
    Description: str = Field(..., min_length=1)

def get_db():
    client = MongoClient(MONGODB_URI)
    return client[DATABASE_NAME][COLLECTION_NAME]

@asynccontextmanager
async def lifespan(app: FastAPI):
    # Startup and check MongoDB connection
    try:
        get_db().find_one()
    except Exception as e:
        print(f"Warning: MongoDB connection failed: {e}")
    yield
    # Shutdown
    pass

app = FastAPI(
    title="Inventory Management API",
```

```
    description="Complete API for product inventory management",
    version="1.0.0",
    lifespan=lifespan,
)

# Middleware for Prometheus metrics
@app.middleware("http")
async def metrics_middleware(request, call_next):
    import time
    start = time.time()
    response = await call_next(request)
    duration = time.time() - start
    REQUEST_COUNT.labels(
        method=request.method,
        endpoint=request.url.path,
        status=response.status_code
    ).inc()
    REQUEST_LATENCY.labels(endpoint=request.url.path).observe(duration)
    return response

@app.get("/getSingleProduct")
async def get_single_product(
    id: int = Query(..., gt=0, description="Product ID")
):
    """Return a single product by ID"""
    product = get_db().find_one({"ProductID": id})
    if not product:
        raise HTTPException(status_code=404, detail="Product not found")
    product["_id"] = str(product["_id"])
```

```
    return product


@app.get("/getAll")
async def get_all():
    """Return all products in the inventory"""
    products = list(get_db().find({}))
    for p in products:
        p["_id"] = str(p["_id"])
    return products


@app.post("/addNew")
async def add_new(product: ProductCreate):
    """Add a new product to the inventory"""
    db = get_db()
    if db.find_one({"ProductID": product.ProductID}):
        raise HTTPException(status_code=400, detail="Product ID already exists")
    doc = product.model_dump()
    db.insert_one(doc)
    return {"message": "Product added successfully", "ProductID": product.ProductID}


@app.delete("/deleteOne")
async def delete_one(
    id: int = Query(..., gt=0, description="Product ID to delete")
):
    """Delete a product by ID"""
    result = get_db().delete_one({"ProductID": id})
    if result.deleted_count == 0:
        raise HTTPException(status_code=404, detail="Product not found")
    return {"message": "Product deleted successfully"}
```

```
@app.get("/startsWith")
async def starts_with(
    letter: str = Query(..., min_length=1, max_length=1, description="Letter to filter by")
):
    """Return all products whose name starts with the given letter"""
    letter = letter.upper()
    products = list(get_db().find({
        "Name": {"$regex": f"^{letter}", "$options": "i"}
    }))
    for p in products:
        p["_id"] = str(p["_id"])
    return products


@app.get("/paginate")
async def paginate(
    start_id: int = Query(..., gt=0, alias="startId", description="Starting product ID"),
    end_id: int = Query(..., gt=0, alias="endId", description="Ending product ID")
):
    """Return products in batches of 10 between start and end ID"""
    if start_id > end_id:
        raise HTTPException(status_code=400, detail="startId must be <= endId")
    products = list(get_db().find(
        {"ProductID": {"$gte": start_id, "$lte": end_id}}
    ).sort("ProductID", 1).limit(10))
    for p in products:
        p["_id"] = str(p["_id"])
    return products
```

```
@app.get("/convert")
async def convert(
    id: int = Query(..., gt=0, description="Product ID to get price in EUR")
):
    """Return the product's price converted from USD to EUR"""
    product = get_db().find_one({"ProductID": id})
    if not product:
        raise HTTPException(status_code=404, detail="Product not found")

    async with httpx.AsyncClient() as client:
        resp = await client.get(
            "https://api.frankfurter.dev/v1/latest?base=USD&symbols=EUR"
        )
        if resp.status_code != 200:
            raise HTTPException(status_code=502, detail="Exchange rate API unavailable")
        data = resp.json()
        rate = data["rates"]["EUR"]

    price_usd = product["UnitPrice"]
    price_eur = round(price_usd * rate, 2)
    return {
        "ProductID": id,
        "Name": product["Name"],
        "PriceUSD": price_usd,
        "PriceEUR": price_eur,
        "ExchangeRate": rate,
    }

# Prometheus metrics endpoint for API monitoring
```

```
@app.get("/metrics")
async def metrics():
    """Prometheus metrics for API performance monitoring"""
    return Response(
        content=generate_latest(),
        media_type=CONTENT_TYPE_LATEST
    )

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

import_csv_to_mongodb.py:

```
import csv
import json
import os
from pymongo import MongoClient

# MongoDB connection
MONGODB_URI = os.getenv("MONGODB_URI", "mongodb://localhost:27017")
DATABASE_NAME = "inventory_db"
COLLECTION_NAME = "products"
CSV_FILE = "products.csv"

def convert_csv_to_json(csv_path: str) -> list[dict]:
    """Read CSV and convert to list of JSON-compatible dicts"""
    products = []
    with open(csv_path, "r", encoding="utf-8") as f:
        reader = csv.DictReader(f)
```

```
        for row in reader:
            product = {
                "ProductID": int(row["ProductID"]),
                "Name": row["Name"],
                "UnitPrice": float(row["UnitPrice"]),
                "StockQuantity": int(row["StockQuantity"]),
                "Description": row["Description"],
            }
            products.append(product)
    return products


def import_to_mongodb(products: list[dict]) -> None:
    """Insert products into MongoDB, replacing existing collection"""
    client = MongoClient(MONGODB_URI)
    db = client[DATABASE_NAME]
    collection = db[COLLECTION_NAME]

    # Clear existing data and insert fresh
    collection.delete_many({})
    collection.insert_many(products)

    # Create index on ProductID for fast lookups
    collection.create_index("ProductID", unique=True)

    print(f"Successfully imported {len(products)} products to MongoDB")
    client.close()


def main():
    script_dir = os.path.dirname(os.path.abspath(__file__))
```

```
    csv_path = os.path.join(script_dir, CSV_FILE)

    if not os.path.exists(csv_path):
        print(f"Error: {CSV_FILE} not found at {csv_path}")
        return

    products = convert_csv_to_json(csv_path)
    import_to_mongodb(products)

if __name__ == "__main__":
    main()
```

Unit Test Code

Postman_collection.json:

```
{
  "info": {
    "name": "Inventory API Tests",
    "schema": "https://schema.getpostman.com/json/collection/v2.1.0/collection.json"
  },
  "item": [
    {
      "name": "Test getSingleProduct",
      "event": [
        {
          "listen": "test",
          "script": {
            "type": "text/javascript",
            "exec": [
```

```
        "pm.test('Status code is 200', function () {",
        "    pm.response.to.have.status(200);",
        "});",
        "pm.test('Response is JSON', function () {",
        "    pm.response.to.be.json;",
        "});",
        "pm.test('Product has correct ID', function () {",
        "    var jsonData = pm.response.json();",
        "    pm.expect(jsonData.ProductID).to.eql(1001);",
        "});",
        "pm.test('Product has all required fields', function () {",
        "    var jsonData = pm.response.json();",
        "    pm.expect(jsonData).to.have.property('ProductID');",
        "    pm.expect(jsonData).to.have.property('Name');",
        "    pm.expect(jsonData).to.have.property('UnitPrice');",
        "    pm.expect(jsonData).to.have.property('StockQuantity');",
        "    pm.expect(jsonData).to.have.property('Description');",
        "});"
      ]
    }
   }
  ],
  "request": {
   "method": "GET",
   "url": "{{base_url}}/getSingleProduct?id=1001"
  }
 },
 {
  "name": "Test getSingleProduct invalid ID returns 404",
```

```
  "event": [
    {
      "listen": "test",
      "script": {
        "type": "text/javascript",
        "exec": [
          "pm.test('Status code is 404 for non-existent product', function () {",
          "    pm.response.to.have.status(404);",
          "});",
          "pm.test('Error detail is returned', function () {",
          "    var jsonData = pm.response.json();",
          "    pm.expect(jsonData).to.have.property('detail');",
          "});"
        ]
      }
    }
  ],
  "request": {
    "method": "GET",
    "url": "{{base_url}}/getSingleProduct?id=99999"
  }
},
{
  "name": "Test getAll",
  "event": [
    {
      "listen": "test",
      "script": {
        "type": "text/javascript",
```

```
      "exec": [
        "pm.test('Status code is 200', function () {",
        "    pm.response.to.have.status(200);",
        "});",
        "pm.test('Response is a non-empty array', function () {",
        "    var jsonData = pm.response.json();",
        "    pm.expect(jsonData).to.be.an('array');",
        "    pm.expect(jsonData.length).to.be.above(0);",
        "});",
        "pm.test('Each product has required fields', function () {",
        "    var jsonData = pm.response.json();",
        "    jsonData.slice(0, 5).forEach(function (product) {",
        "        pm.expect(product).to.have.property('ProductID');",
        "        pm.expect(product).to.have.property('Name');",
        "        pm.expect(product).to.have.property('UnitPrice');",
        "        pm.expect(product).to.have.property('StockQuantity');",
        "        pm.expect(product).to.have.property('Description');",
        "    });",
        "});"
      ]
    }
   }
  ],
  "request": {
   "method": "GET",
   "url": "{{base_url}}/getAll"
  }
 },
 {
```

```
    "name": "Test addNew",
    "event": [
     {
      "listen": "test",
      "script": {
       "type": "text/javascript",
       "exec": [
        "pm.test('Status code is 200', function () {",
        "    pm.response.to.have.status(200);",
        "});",
        "pm.test('Success message returned', function () {",
        "    var jsonData = pm.response.json();",
        "    pm.expect(jsonData.message).to.eql('Product added successfully');",
        "    pm.expect(jsonData.ProductID).to.eql(9999);",
        "});"
       ]
      }
     }
    ],
    "request": {
     "method": "POST",
     "url": "{{base_url}}/addNew",
     "header": [{"key": "Content-Type", "value": "application/json"}],
     "body": {
      "mode": "raw",
      "raw": "{\"ProductID\": 9999, \"Name\": \"Test Product\", \"UnitPrice\": 99.99,
\"StockQuantity\": 10, \"Description\": \"Test description for unit testing\"}"
     }
    }
   },
```

```
  {
    "name": "Test addNew duplicate returns 400",
    "event": [
      {
        "listen": "test",
        "script": {
          "type": "text/javascript",
          "exec": [
            "pm.test('Status code is 400 for duplicate ID', function () {",
            "    pm.response.to.have.status(400);",
            "});",
            "pm.test('Error detail mentions already exists', function () {",
            "    var jsonData = pm.response.json();",
            "    pm.expect(jsonData.detail).to.include('already exists');",
            "});"
          ]
        }
      }
    ],
    "request": {
      "method": "POST",
      "url": "{{base_url}}/addNew",
      "header": [{"key": "Content-Type", "value": "application/json"}],
      "body": {
        "mode": "raw",
        "raw": "{\"ProductID\": 9999, \"Name\": \"Duplicate Product\", \"UnitPrice\": 10.00, \"StockQuantity\": 1, \"Description\": \"Should fail as duplicate\"}"
      }
    }
  },
```

```
{
  "name": "Test deleteOne",
  "event": [
    {
      "listen": "test",
      "script": {
        "type": "text/javascript",
        "exec": [
          "pm.test('Status code is 200', function () {",
          "    pm.response.to.have.status(200);",
          "});",
          "pm.test('Delete success message returned', function () {",
          "    var jsonData = pm.response.json();",
          "    pm.expect(jsonData.message).to.eql('Product deleted successfully');",
          "});"
        ]
      }
    }
  ],
  "request": {
    "method": "DELETE",
    "url": "{{base_url}}/deleteOne?id=9999"
  }
},
{
  "name": "Test deleteOne non-existent returns 404",
  "event": [
    {
      "listen": "test",
```

```
      "script": {
        "type": "text/javascript",
        "exec": [
          "pm.test('Status code is 404 for already deleted product', function () {",
          "    pm.response.to.have.status(404);",
          "});"
        ]
      }
    }
  ],
  "request": {
    "method": "DELETE",
    "url": "{{base_url}}/deleteOne?id=9999"
  }
},
{
  "name": "Test startsWith",
  "event": [
    {
      "listen": "test",
      "script": {
        "type": "text/javascript",
        "exec": [
          "pm.test('Status code is 200', function () {",
          "    pm.response.to.have.status(200);",
          "});",
          "pm.test('Response is an array', function () {",
          "    var jsonData = pm.response.json();",
          "    pm.expect(jsonData).to.be.an('array');",
```

```
        "});",
        "pm.test('All returned products start with S', function () {",
        "    var jsonData = pm.response.json();",
        "    jsonData.forEach(function (product) {",
        "        pm.expect(product.Name.charAt(0).toUpperCase()).to.eql('S');",
        "    });",
        "});"
       ]
     }
    }
  ],
  "request": {
    "method": "GET",
    "url": "{{base_url}}/startsWith?letter=s"
  }
},
{
  "name": "Test paginate",
  "event": [
    {
      "listen": "test",
      "script": {
        "type": "text/javascript",
        "exec": [
          "pm.test('Status code is 200', function () {",
          "    pm.response.to.have.status(200);",
          "});",
          "pm.test('Response is an array', function () {",
          "    var jsonData = pm.response.json();",
```

```
        "    pm.expect(jsonData).to.be.an('array');",
        "});",
        "pm.test('Returns at most 10 products', function () {",
        "    var jsonData = pm.response.json();",
        "    pm.expect(jsonData.length).to.be.at.most(10);",
        "});",
        "pm.test('Products are within the requested ID range', function () {",
        "    var jsonData = pm.response.json();",
        "    jsonData.forEach(function (product) {",
        "        pm.expect(product.ProductID).to.be.at.least(1001);",
        "        pm.expect(product.ProductID).to.be.at.most(1020);",
        "    });",
        "});"
      ]
    }
   }
  ],
  "request": {
   "method": "GET",
   "url": "{{base_url}}/paginate?startId=1001&endId=1020"
  }
 },
 {
  "name": "Test convert",
  "event": [
   {
    "listen": "test",
    "script": {
     "type": "text/javascript",
```

```
      "exec": [
        "pm.test('Status code is 200', function () {",
        "    pm.response.to.have.status(200);",
        "});",
        "pm.test('Response contains USD and EUR prices', function () {",
        "    var jsonData = pm.response.json();",
        "    pm.expect(jsonData).to.have.property('PriceUSD');",
        "    pm.expect(jsonData).to.have.property('PriceEUR');",
        "    pm.expect(jsonData).to.have.property('ExchangeRate');",
        "});",
        "pm.test('EUR price is less than USD price', function () {",
        "    var jsonData = pm.response.json();",
        "    pm.expect(jsonData.PriceEUR).to.be.below(jsonData.PriceUSD);",
        "});",
        "pm.test('Exchange rate is a positive number', function () {",
        "    var jsonData = pm.response.json();",
        "    pm.expect(jsonData.ExchangeRate).to.be.above(0);",
        "});"
      ]
    }
   }
  ],
  "request": {
   "method": "GET",
   "url": "{{base_url}}/convert?id=1001"
  }
 },
 {
  "name": "Test metrics endpoint",
```

```
    "event": [
      {
        "listen": "test",
        "script": {
          "type": "text/javascript",
          "exec": [
            "pm.test('Status code is 200', function () {",
            "    pm.response.to.have.status(200);",
            "});",
            "pm.test('Response contains Prometheus metrics', function () {",
            "    pm.expect(pm.response.text()).to.include('api_requests_total');",
            "    pm.expect(pm.response.text()).to.include('api_request_duration_seconds');",
            "});"
          ]
        }
      }
    ],
    "request": {
      "method": "GET",
      "url": "{{base_url}}/metrics"
    }
  }
 ],
 "variable": [
  {"key": "base_url", "value": "http://localhost:8000"}
 ]
}
```

Docker code

Dockerfile:

```
FROM ubuntu:22.04

ENV DEBIAN_FRONTEND=noninteractive
RUN apt-get update && apt-get install -y \
    python3 \
    python3-pip \
    python3-venv \
    git \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

# Copy application files
COPY requirements.txt .
RUN pip3 install --no-cache-dir -r requirements.txt
COPY main.py .
COPY products.csv .

# Import script for initial data load
COPY import_csv_to_mongodb.py .

EXPOSE 8000

CMD ["python3", "-m", "uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

docker-compose.yml

```
services:
  mongodb:
    image: mongo:7
    volumes:
      - mongodb_data:/data/db

  api:
    build: .
    ports:
      - "${API_HOST_PORT:-8000}:8000"
    environment:
      - MONGODB_URI=mongodb://mongodb:27017
    depends_on:
      - mongodb
    command: >
      sh -c "python3 import_csv_to_mongodb.py &&
             python3 -m uvicorn main:app --host 0.0.0.0 --port 8000"

  prometheus:
    profiles: ["monitoring"]
    image: prom/prometheus:latest
    ports:
      - "${PROMETHEUS_HOST_PORT:-9090}:9090"
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml
    command:
      - '--config.file=/etc/prometheus/prometheus.yml'
      - '--web.enable-lifecycle'
    depends_on:
```

```
      - api

  grafana:
    profiles: ["monitoring"]
    image: grafana/grafana:latest
    ports:
      - "${GRAFANA_HOST_PORT:-3000}:3000"
    environment:
      - GF_SECURITY_ADMIN_PASSWORD=admin
      - GF_USERS_ALLOW_SIGN_UP=false
    depends_on:
      - prometheus

volumes:
  mongodb_data:
```

prometheus.yml

```
# Prometheus config for API monitoring
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: 'inventory-api'
    static_configs:
      - targets: ['api:8000']
    metrics_path: '/metrics'
```